ADT Lab 10 (10pts)

Neo4j

-- Shivali Mate

```
!pip install py2neo
!pip install pandas
!pip install scikit-learn
```

```
Collecting py2neo
  Downloading py2neo-2021.2.4-py2.py3-none-any.whl.metadata (9.9 kB)
Requirement already satisfied: certifi in c:\users\shiva\appdata\
local\programs\python\python312\lib\site-packages (from py2neo)
(2025.10.5)
Collecting interchange~=2021.0.4 (from py2neo)
  Downloading interchange-2021.0.4-py2.py3-none-any.whl.metadata (1.9
kB)
Collecting monotonic (from py2neo)
  Downloading monotonic-1.6-py2.py3-none-any.whl.metadata (1.5 kB)
Requirement already satisfied: packaging in c:\users\shiva\appdata\
local\programs\python\python312\lib\site-packages (from py2neo) (24.1)
Collecting pansi>=2020.7.3 (from py2neo)
  Downloading pansi-2024.11.0-py2.py3-none-any.whl.metadata (3.1 kB)
Requirement already satisfied: pygments>=2.0.0 in c:\users\shiva\
appdata\local\programs\python\python312\lib\site-packages (from
py2neo) (2.18.0)
Requirement already satisfied: six>=1.15.0 in c:\users\shiva\appdata\
local\programs\python\python312\lib\site-packages (from py2neo)
(1.16.0)
Requirement already satisfied: urllib3 in c:\users\shiva\appdata\
local\programs\python\python312\lib\site-packages (from py2neo)
(2.2.2)
Requirement already satisfied: pytz in c:\users\shiva\appdata\local\
programs\python\python312\lib\site-packages (from
interchange~=2021.0.4->py2neo) (2024.2)
Requirement already satisfied: pillow in c:\users\shiva\appdata\local\
programs\python\python312\lib\site-packages (from pansi>=2020.7.3-
>py2neo) (10.4.0)
Downloading py2neo-2021.2.4-py2.py3-none-any.whl (177 kB)
Downloading interchange-2021.0.4-py2.py3-none-any.whl (28 kB)
Downloading pansi-2024.11.0-py2.py3-none-any.whl (26 kB)
Downloading monotonic-1.6-py2.py3-none-any.whl (8.2 kB)
Installing collected packages: monotonic, pansi, interchange, py2neo

   ---------- ----------------------------- 1/4 [pansi]
   ---------- ----------------------------- 1/4 [pansi]
```

```
   ------------------- ------------------- 2/4 [interchange]
   ------------------- ------------------- 2/4 [interchange]
   ------------------------------ --------- 3/4 [py2neo]
   ------------------------------ --------- 3/4 [py2neo]
   ------------------------------ --------- 3/4 [py2neo]
   ------------------------------ --------- 3/4 [py2neo]
   ------------------------------ --------- 3/4 [py2neo]
   ------------------------------ --------- 3/4 [py2neo]
   ------------------------------ --------- 3/4 [py2neo]
   ---------------------------------------- 4/4 [py2neo]

Successfully installed interchange-2021.0.4 monotonic-1.6 pansi-
2024.11.0 py2neo-2021.2.4


[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip

Requirement already satisfied: pandas in c:\users\shiva\appdata\local\
programs\python\python312\lib\site-packages (2.2.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\shiva\
appdata\local\programs\python\python312\lib\site-packages (from
pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\
shiva\appdata\local\programs\python\python312\lib\site-packages (from
pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\shiva\appdata\
local\programs\python\python312\lib\site-packages (from pandas)
(2024.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\shiva\
appdata\local\programs\python\python312\lib\site-packages (from
pandas) (2024.2)
Requirement already satisfied: six>=1.5 in c:\users\shiva\appdata\
local\programs\python\python312\lib\site-packages (from python-
dateutil>=2.8.2->pandas) (1.16.0)


[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip

Requirement already satisfied: scikit-learn in c:\users\shiva\appdata\
local\programs\python\python312\lib\site-packages (1.5.2)
Requirement already satisfied: numpy>=1.19.5 in c:\users\shiva\
appdata\local\programs\python\python312\lib\site-packages (from
scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in c:\users\shiva\appdata\
local\programs\python\python312\lib\site-packages (from scikit-learn)
(1.14.1)
Requirement already satisfied: joblib>=1.2.0 in c:\users\shiva\
appdata\local\programs\python\python312\lib\site-packages (from
```

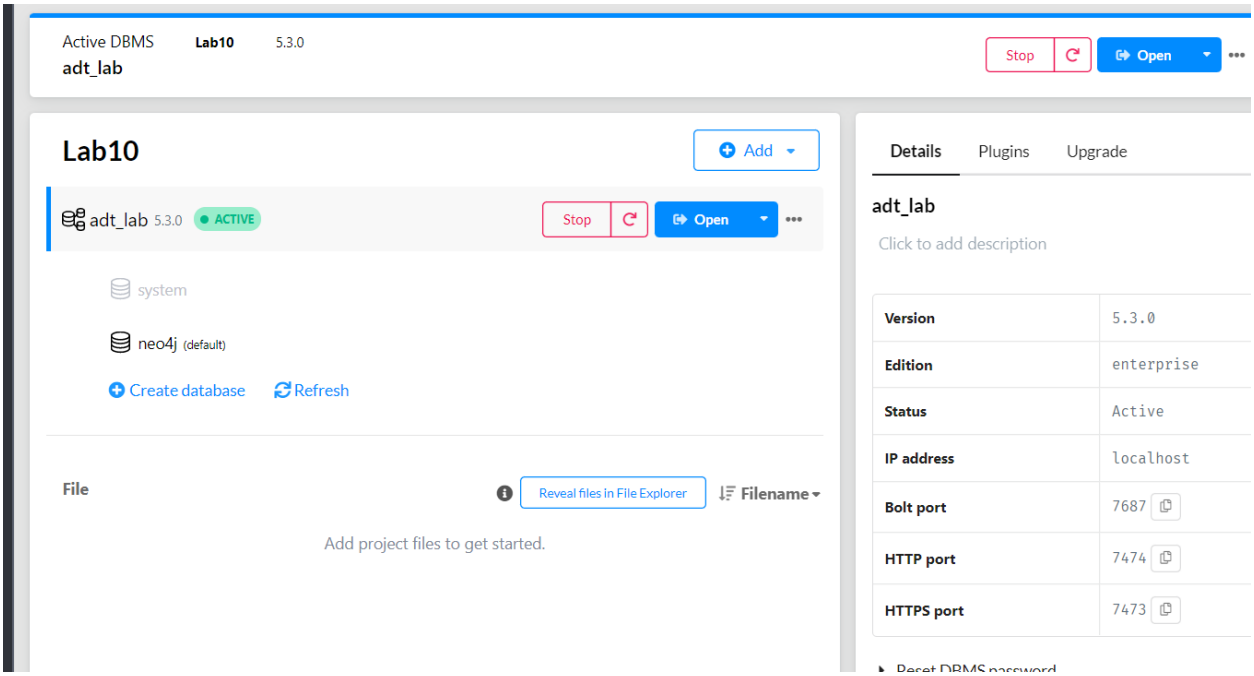
```
scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\shiva\
appdata\local\programs\python\python312\lib\site-packages (from
scikit-learn) (3.5.0)


[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Setup Instructions

1. Create a new project in Neo4j desktop
2. Click on Add button
3. Click Local DBMS
4. Set your dbms name and password. (For simplicity keep password as password)
5. Start the db

Your neo4j screen should look something like this-



1. Once the database is started run the below cells to populate the newly created neo4j database

```
import os
import pandas as pd
from py2neo import Graph, Node, Relationship
from sklearn.metrics.pairwise import cosine_similarity

# Load the ratings data
ratings = pd.read_csv(os.getcwd()+'/ratings.csv')
ratings=ratings.loc[:10000]
```

```
# Load the movie data
movies = pd.read_csv(os.getcwd()+'/movies.csv')
movies=movies[movies['movieId'].isin(ratings['movieId'].unique().tolis
t())].reset_index()

# Connect to Neo4j
graph = Graph('bolt://localhost:7687', auth=('neo4j', 'password'))
graph.run("CREATE CONSTRAINT FOR (u:User) REQUIRE u.userId IS UNIQUE")
graph.run("CREATE CONSTRAINT FOR (m:Movie) REQUIRE m.movieId IS
UNIQUE")
```

```
(No data)
```

```
d={}
users_d = {}
movies_d = {}

# Create movie nodes
for index, movie in movies.iterrows():
    node = Node('Movie',
movieId=int(movie['movieId']),title=movie['title'])
    graph.merge(node, "Movie", "movieId")

    movies_d[movie['movieId']]=node
    d[movie['movieId']]=movie['title']

# Create user nodes
for userId in ratings['userId'].unique():
    node = Node('User', userId=int(userId))
    graph.merge(node, "User", "userId")

    users_d[int(userId)]=node

# Create rating relationships
for index, rating in ratings.iterrows():
    rating_float = float(rating['rating'])
    id_int = int(rating['userId'])
    movie_id_int = int(rating['movieId'])

    user_node = users_d[id_int]
    movie_node = movies_d[movie_id_int]

    rel = Relationship(user_node, 'RATED', movie_node,
rating=rating_float)
    graph.create(rel)
```

Now check your Neo4j browser. Your data should reflect there-

19	703
28	570
64	517
18	502
57	476
21	443
42	440
45	399
62	366
51	359

Create queries for the below questions and run it

Example Query 1

Find the top-rated movies (only top 10 results)

```
query = f"MATCH (m:Movie)<-[r:RATED]-() RETURN m.title, AVG(r.rating)
AS avg_rating ORDER BY avg_rating DESC LIMIT 10"

results = graph.run(query)

for record in results:
    print(record)
```

```
'Three Colors: Red (Trois couleurs: Rouge) (1994)'    5.0
'Murder in the First (1995)'    5.0
'Canadian Bacon (1995)'    5.0
'Living in Oblivion (1995)'5.0
'Persuasion (1995)'   5.0
"Antonia's Line (Antonia) (1995)"    5.0
'Home for the Holidays (1995)'   5.0
'Bottle Rocket (1996)'    5.0
'Flirting With Disaster (1996)'  5.0
'Farewell My Concubine (Ba wang bie ji) (1993)'  5.0
```

This query finds the top-rated movies in the dataset by calculating the average rating for each movie and sorting the results in descending order.

Query 2

Find the top 10 users who have rated the most movies

```
query = f"MATCH (u:User)-[r:RATED]->(m:Movie) RETURN u.userId AS
userId, COUNT(r) AS ratingCount ORDER BY ratingCount DESC LIMIT 10"
results = graph.run(query)
for record in results: print(record)
```

```
19    703
28    570
64    517
18    502
57    476
21    443
42    440
45    399
62    366
51    359
```

This query finds the users who have rated the most movies in the dataset by counting the number of movies each user has rated and sorting the results in descending order.

Hint-

- Use COUNT() aggregate clause

Expected result-

```
19      703
28      570
64      517
18      502
57      476
21      443
42      440
45      399
62      366
51      359
```

Query 3

Find top 10 movies that were rated the highest by a the user 2

```
query = """ MATCH (u:User {userId: 2} )-[rated:RATED]->(movie:Movie)
WITH movie, MAX(rated.rating) AS maxRating ORDER BY maxRating DESC
RETURN movie.title AS Movie, maxRating AS Rating LIMIT 10;"""
results = graph.run(query)
for record in results: print(record)
```

```
'Step Brothers (2008)'     5.0
'Inside Job (2010)'   5.0
'The Jinx: The Life and Deaths of Robert Durst (2015)'5.0
'Warrior (2011)'5.0
'Mad Max: Fury Road (2015)'5.0
'Wolf of Wall Street, The (2013)'     5.0
'Inglourious Basterds (2009)'    4.5
```

```
'Good Will Hunting (1997)' 4.5
'Dark Knight, The (2008)'  4.5
'Town, The (2010)'    4.5
```

Hint-

- Use MAX() aggregate clause

Expected result-

```
'Mad Max: Fury Road (2015)'     5.0
'The Jinx: The Life and Deaths of Robert Durst (2015)'  5.0
'Warrior (2011)'        5.0
'Inside Job (2010)'     5.0
'Step Brothers (2008)'  5.0
'Wolf of Wall Street, The (2013)'       5.0
'Dark Knight, The (2008)'       4.5
'Town, The (2010)'      4.5
'Inglourious Basterds (2009)'   4.5
'Good Will Hunting (1997)'      4.5
```

Query 4

Find the average rating for the moiveId 356

```
query = ''' MATCH (movie:Movie {movieId: 356} )<-[rated:RATED]-
(user:User) WITH movie, AVG(rated.rating) AS averageRating RETURN
movie.title AS Movie, averageRating AS AverageRating'''
results = graph.run(query)
for record in results: print(record)

'Forrest Gump (1994)' 4.1842105263157885
```

Hint-

- Use AVG() aggregate clause

Expected result-

```
'Forrest Gump (1994)'   4.1842105263157885
```

Run the queries, convert into pdf and submit it

Optional

```
# A function to get movie recommendations for a user
def get_movie_recommendations(userId):
    query = f"MATCH (u:User {{userId: {userId}}})-[:RATED]-
>(m:Movie)<-[:RATED]-(u2:User)-[r2:RATED]->(m2:Movie) WHERE NOT (u)-
[:RATED]->(m2) RETURN m2.title AS title, AVG(r2.rating) AS rating
ORDER BY rating DESC LIMIT 10"
    results = graph.run(query)

    # Convert the results to a pandas DataFrame
    recommendations = pd.DataFrame(results, columns=['title',
'rating'])

    return recommendations

# Get recommendations for user 3
recommendations = get_movie_recommendations(3)
print(recommendations)
```

```
                                        title  rating
0                      Wolf Man, The (1941)     5.0
1  Winnie the Pooh and the Blustery Day (1968)     5.0
2       All Quiet on the Western Front (1930)     5.0
3                           SLC Punk! (1998)     5.0
4         Mr. Smith Goes to Washington (1939)     5.0
5                      Bottle Rocket (1996)     5.0
6                  Newton Boys, The (1998)     5.0
7                Pink Floyd: The Wall (1982)     5.0
8                     Canadian Bacon (1995)     5.0
9                  Gulliver's Travels (1939)     5.0
```

- The recommendation algorithm above is a simple content-based filtering algorithm, which makes recommendations based on the similarity between items (in this case, movies).
- In a content-based filtering algorithm, the system makes recommendations based on the attributes or features of the items that the user has previously interacted with. In the above example, the system calculates the cosine similarity between the movie that the user has rated and all other movies in the dataset based on the genres of the movies, and recommends the top N most similar movies.

Understand the above query and write what you understood. How the recommendation algorithm is working?

The given query creates a simple movie recommendation system based on user similarity. It works by finding other users who have rated the same movies as the selected user and then identifying additional movies those similar users have rated that the original user hasn't seen yet. Essentially, it assumes that if two users share similar tastes in movies, the unseen movies rated highly by one user could be good recommendations for the other. This approach is a form

of collaborative filtering, where recommendations are made using relationships between users and their movie ratings stored in the Neo4j graph database.